A* Pathfinding Simulator: Multi-Agent Collaborative Pathfinding with Real-Time Visualization

Cameron Grenier
169069484
Wilfrid Laurier University
Ontario, Canada

***Abstract*—This paper presents a multi-agent A* pathfinding simulator implemented in C, where N agents concurrently navigate a shared 2D grid under fog of war with collision avoidance. The system employs a zone-based concurrency model using POSIX threads, partitioning the map into overlapping rectangular zones with per-zone mutexes for fine-grained locking and deadlock-free operation via global lock ordering. Agents coordinate through a shared obstacle map and a tuned cost penalty for agent-occupied cells, providing decentralized livelock mitigation without centralized planning. A companion comparison report details the full benchmark results; in summary, 3,200 runs across grid sizes from 10×10 to 70×70 with up to 8 agents achieve a 100% success rate with 3–12% path-length overhead over omniscient BFS.**

***Index Terms*—A* algorithm, multi-agent pathfinding, collision avoidance, POSIX threads, zone-based locking**

I. Introduction

This project implements a multi-agent A* pathfinding simulator in C. N agents concurrently navigate a shared 2D grid toward a common goal under fog of war (obstacles are discovered at runtime via a dual-map architecture), with collision avoidance (agents detect and route around each other), zone-based concurrency (the map is partitioned into overlapping rectangular zones, each with its own mutex/condvar, for fine-grained locking), real-time ncurses visualization (a dedicated rendering thread with colour-coded cells and camera scrolling), an interactive menu system (file picker, random map generator, speed selector), and post-simulation analytics (elapsed time, per-agent step counts, full path histories).

II. System Architecture

A. Lifecycle

Fig. 1 shows the main thread's execution flow.

B. Modules

Table I lists the source modules.

C. Dual-Map Architecture

The simulator maintains two separate 2D `Node` arrays:

`actual_map` (ground truth)
Built from the input file with all obstacles and neighbour pointers. Read-only after initialization. A* traverses this pointer graph for correct connectivity.

`shared_map` (collective knowledge)
Initialized with only agent positions and the goal visible; all other cells start as `EMPTY`. Obstacles are written here at runtime when an agent discovers them. A* checks `shared_map` types for traversability but follows `actual_map` pointers for neighbours. The `type` field uses `_Atomic NodeType` for safe concurrent reads/writes.

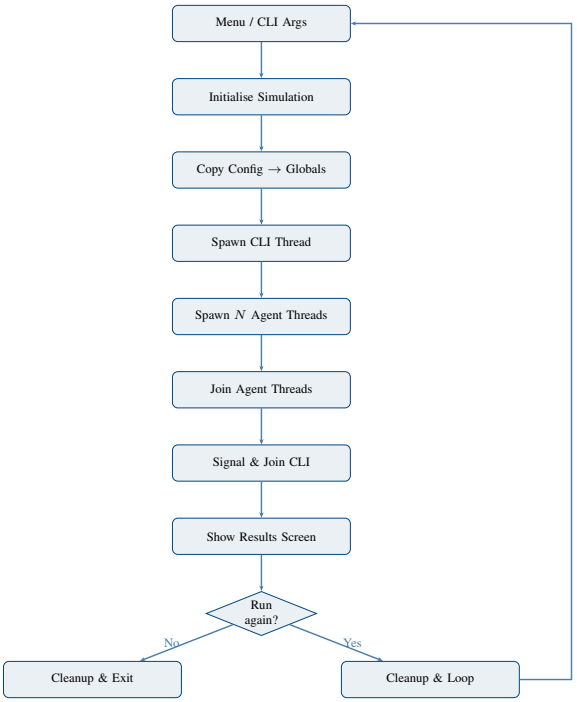

Fig. 1. Main thread lifecycle.

TABLE I
SOURCE MODULES AND THEIR RESPONSIBILITIES.

Module	Header	Responsibilities
`main.c`	–	Entry point; CLI argument parsing (`-speed`); lifecycle orchestration; global variables; `show_results()`; `cleanup_sim()`.
`map.c`	`map.h`	Input parsing; dual-map construction (`actual_map` vs. `shared_map`); neighbour linking; agent/goal placement; zone partitioning with 25% overlap; mutex/cond init per zone.
`astar.c`	`astar.h`	A* search; Manhattan and Euclidean heuristics; min-heap open set; flat `char` closed set; flat `int` g-cost array for duplicate detection; flat `Node*` came-from map; agent-cell penalty; `PathNode` linked-list reconstruction.
`runner.c`	`runner.h`	Agent thread entry (`runner()`); zone collection, dedup, sorted locking; collision detection/wait; path validation; obstacle discovery and broadcast; atomic map updates; path history; CLI signal; speed delay.
`cli.c`	`cli.h`	Rendering thread; ncurses with colour pairs; adaptive cell-width (3/2/1); camera scrolling; `pthread_cond_timedwait` signal-driven redraws.
`menu.c`	`menu.h`	Interactive ncurses menus; file picker (`opendir`/`readdir`); custom path input; random map generator; speed selector.
`heap.c`	`heap.h`	Min-heap for A* open set; dynamic resize (double on full, halve below 25%).
`types.c`	`types.h`	Core structs (`NodeType`, `Coords`, `Node`, `Agent`, `Zone`, `SimConfig`); `AStarNode`, `PathNode` in `astar.h`; `MinHeap` in `heap.h`.

III. A* ALGORITHM

A. Data Structures

```
1  typedef struct {
2      Node *node;   /* Pointer to node in actual_map */
3      int g;        /* Cost from start to this node */
4      int f;        /* Estimated total cost: g + h */
5  } AStarNode;
6
7  typedef struct PathNode {
8      Node *node;            /* Pointer to node in actual_map */
9      struct PathNode *next; /* Next node in path toward goal */
10 } PathNode;
```

Listing 1. AStarNode (astar.h:11–16) and PathNode (astar.h:21–25).

The open set is a binary min-heap of `AStarNode` values ordered by `f` (initial capacity 32, doubles on full, halves below 25%). The closed set is a flat `char` array (`rows` $\times$ `cols`, zero-initialized) giving $\mathcal{O}(1)$ lookup at 1 byte per cell. The `g_cost_array` is a flat `int` array initialized to `INT_MAX`, tracking the best known g-cost to each cell for duplicate detection (see Section III-C). The came-from map is a flat `Node*` array indexed identically, enabling $\mathcal{O}(L)$ path reconstruction.

TABLE II
HEAP OPERATION COMPLEXITY.

Operation	Time	Amortized
`heap_push`	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`heap_pop`	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
`heap_create`	$\mathcal{O}(1)$	–
Min access	$\mathcal{O}(1)$	–

B. Heuristic Functions

Two heuristics are supported, selectable at compile time:

$$h_M(n) = |n_x - g_x| + |n_y - g_y| \quad \text{(Manhattan)} \tag{1}$$

$$h_E(n) = \sqrt{(n_x - g_x)^2 + (n_y - g_y)^2} \quad \text{(Euclidean)} \tag{2}$$

Both are admissible and consistent on a 4-connected grid. Manhattan is the tighter bound ($h_E(n) \leq h_M(n)$ for all n), so A* expands fewer nodes and runs faster. Euclidean's looser bound explores a wider frontier, yielding lower path-length overhead in multi-agent scenarios at the cost of increased computation time. The companion comparison report quantifies this trade-off across 3,200 benchmark runs.

C. Duplicate Detection

A `g_cost_array` of size rows $\times$ cols, initialized to `INT_MAX`, tracks the lowest g-cost found for each cell. Before pushing a neighbor onto the open set, the algorithm computes a tentative g and compares it against the stored value. If the tentative cost is not strictly less, the neighbor is skipped; otherwise the stored cost, `came_from` pointer, and heap entry are all updated together. This ensures that the `came_from` map always reflects the cheapest known predecessor for every cell, preventing path corruption from stale heap entries.

Additionally, nodes already in the closed set are skipped immediately after popping from the heap. Because the heap may contain multiple entries for the same cell (pushed at different g-costs), this check avoids redundant expansion of previously visited nodes.

D. Search Procedure

Algorithm 1 A* Search with Multi-Agent Modifications

Require: `start_node`,
globals: `goal_node`, `shared_map`, `rows`, `cols`
Ensure: `PathNode*` list from next-move to goal, or `NULL`

```
closed ← calloc(rows × cols)
from ← calloc(rows × cols)
g_cost ← fill(rows × cols, INT_MAX)
open ← heap_create(32)
h_0 ← manhattan(start.pos, goal.pos)
g_cost[start] ← 0
push(open, {start, g=0, f=h_0})
while open.size ≠ 0 do
    curr ← pop(open) {lowest f}
    if closed[curr] then
        continue {stale duplicate}
    end if
    if curr.node = goal_node then
        return RECONSTRUCT(curr, from, start)
    end if
    closed[curr.y · cols + curr.x] ← 1
    for all n ∈ {up, down, left, right} do
        if n = NULL ∨ closed[n] then
            continue
        end if
        t ← shared_map[n.y][n.x].type
        if t = OBSTACLE then
            continue
        end if
        g' ← curr.g + 1
        if t = AGENT ∧ n ≠ goal then
            g' ← curr.g + 2 {soft penalty}
        end if
        if g' ≥ g_cost[n] then
            continue {not a better path}
        end if
        g_cost[n] ← g'
        f' ← g' + manhattan(n, goal)
        push(open, {n, g', f'})
        from[n] ← curr
    end for
end while
return NULL {no path found}
```

1) Multi-Agent Modifications:

1) *Dual-map operation:* A* traverses `actual_map` pointers for connectivity but reads `shared_map` types for traversability, so paths are constrained by current collective knowledge.
2) *Agent-cell penalty ($g + 2$):* Cells with type `AGENT` on `shared_map` are traversable but cost 2 instead of 1, softly biasing paths away from other agents while still allowing passage when no alternative exists. This penalty is a deliberate trade-off discussed in Section V.
3) *Goal exception:* The goal cell always costs 1 even if occupied by an agent.
4) *Duplicate detection via g-cost array:* A flat `int` array tracks the best known g-cost per cell. A neighbor is only pushed to the heap and its `came_from` updated when a strictly cheaper path is found (see Section III-C).

Path reconstruction traces backwards through `came_from`, building a linked list from goal to start, then strips the start node so the first `PathNode` is the agent's next move.

E. Complexity

Let $V = \text{rows} \times \text{cols}$.

TABLE III
A* ALGORITHM COMPLEXITY.

Resource	Complexity	Notes
Open set ops	$\mathcal{O}(V \log V)$	$\leq V$ push/pop
Closed set	$\mathcal{O}(V)$ total	$\mathcal{O}(1)$ each
G-cost array	$\mathcal{O}(V)$ total	$\mathcal{O}(1)$ each
Came-from	$\mathcal{O}(V)$ total	$\mathcal{O}(1)$ each
Path recon.	$\mathcal{O}(L)$	L = path len
Total time	$\mathcal{O}(V \log V)$	Heap-dominated
Space	$\mathcal{O}(V)$	All structures

Peak A* memory: closed set (V bytes) + g-cost array ($4V$ bytes) + came-from ($8V$ bytes on 64-bit) + heap ($16V$ bytes max) $\approx 29V$ bytes.

IV. MULTITHREADING

A. Thread Architecture

The simulator uses $N + 2$ threads: 1 main, 1 CLI renderer, and N agent threads. Fig. 2 illustrates the architecture.

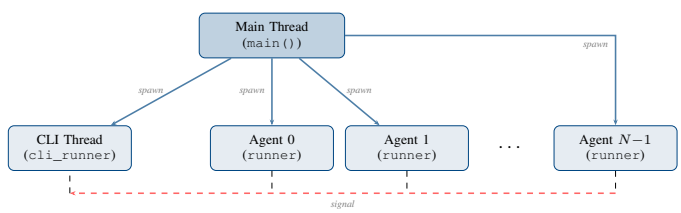

Fig. 2. Thread architecture: 1 main + 1 CLI + N agent threads.

The main thread orchestrates the lifecycle: spawn all threads, join agents, signal and join CLI, display results, cleanup. The CLI thread redraws the grid when signalled by agent moves, using `pthread_cond_timedwait` with a 50ms timeout for keyboard responsiveness. Each agent thread independently runs A*, follows its path, handles obstacles and collisions, and records its history.

B. Zone-Based Locking

The map is divided into rectangular zones of size $\lfloor \text{cols}/5 \rfloor \times \lfloor \text{rows}/5 \rfloor$ (minimum 1), targeting roughly $5 \times 5 = 25$ zones. Each zone has its own mutex and condition variable:

```c
typedef struct {
    int id;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
    Coords top_left;
    Coords bottom_right;
} Zone;
```

Listing 2. Zone structure (types.h:67–75).

1) 25% Overlap: Each zone's bounds are extended by 25% of the zone dimension in every direction (clamped to map edges):

$$\text{overlap}_x = \left\lfloor \frac{\text{zone_size}_x}{4} \right\rfloor, \quad \text{overlap}_y = \left\lfloor \frac{\text{zone_size}_y}{4} \right\rfloor \quad (3)$$

The overlap means an agent near a zone boundary can often lock a single zone covering both its current and next cells, reducing multi-zone lock frequency. It also extends the path validation window beyond the immediate next cell. A position near a boundary may belong to 1, 2, or 4 zones simultaneously.

2) Deadlock Prevention: All threads acquire zone mutexes in ascending zone ID order. Before each move, the agent:

1) Calls `collect_zones()` for all zones overlapping both current and next positions.
2) Deduplicates the zone set.
3) Sorts by `id` ascending (insertion sort, at most 8 elements).
4) Acquires mutexes in ascending order; releases in descending order.

Theorem 1 (Deadlock Freedom): Since all threads acquire zone mutexes in the same total order (ascending zone ID), no circular-wait condition can arise. The timed waits (`pthread_cond_timedwait`) provide additional liveness: even if a signal is missed, agents wake up, release locks, and replan.

```c
static void lock_zones(Zone *zs[], int n) {
    for (int i = 0; i < n; i++)
        pthread_mutex_lock(&zs[i]->mutex);
}
```

Listing 3. Lock acquisition in ascending order (runner.c:133–137).

C. Agent Thread Logic

Fig. 3 shows the per-step decision flow.

1) Collision Detection: When the next cell on `shared_map` is occupied by another agent, the agent releases all held zones except the blocking zone, calls `pthread_cond_timedwait` on that zone's condvar with a speed-scaled timeout, then releases the last lock and replans. The timeout matches `speed_delay` (or 5ms at realtime speed) to prevent both premature replanning and indefinite blocking.

2) Path Validation: After confirming the next cell is not agent-occupied, the runner validates the remaining path within the locked zone region:

```c
PathNode *check = next_move->next;
int valid_path = 1;
while (check != NULL) {
    int in_locked = 0;
    for (int z = 0; z < nlocked; z++) {
        if (within_zone(locked[z], check->node))
            {
            in_locked = 1;
            break;
        }
    }
    if (!in_locked) break;
    NodeType type = shared_map[check->node->pos.y
        ]
                                [check->node->pos.x
                                    ].type;
    if (type == OBSTACLE || type == AGENT) {
        valid_path = 0;
        break;
    }
    check = check->next;
}
```

Listing 4. Path validation (runner.c:244–268).

Validation stops at cells outside the locked zones since those are not protected by held locks. If any cell within the locked region is blocked, the agent unlocks and replans.

3) Obstacle Discovery: If the next cell is `OBSTACLE` on `actual_map` (but `EMPTY` on `shared_map`), the agent writes `OBSTACLE` to `shared_map`, unlocks all zones, broadcasts `map_changed_cond` to wake all agents, and replans. This broadcast is the primary collaborative communication channel.

4) Move Execution: On a valid move, both source and destination cells are updated under all relevant zone locks:

```c
Node *next_node = next_move->node;
shared_map[current_node->pos.y]
          [current_node->pos.x].type = EMPTY;
if (nodes_equal(next_node, goal_node)) {
    shared_map[next_node->pos.y]
              [next_node->pos.x].type = GOAL;
} else {
    shared_map[next_node->pos.y]
              [next_node->pos.x].type = AGENT;
}
```

Listing 5. Atomic map update (runner.c:291–300).

No other agent can observe a partially-completed move. Two agents never occupy the same cell because the check and update are performed atomically under the same locks.

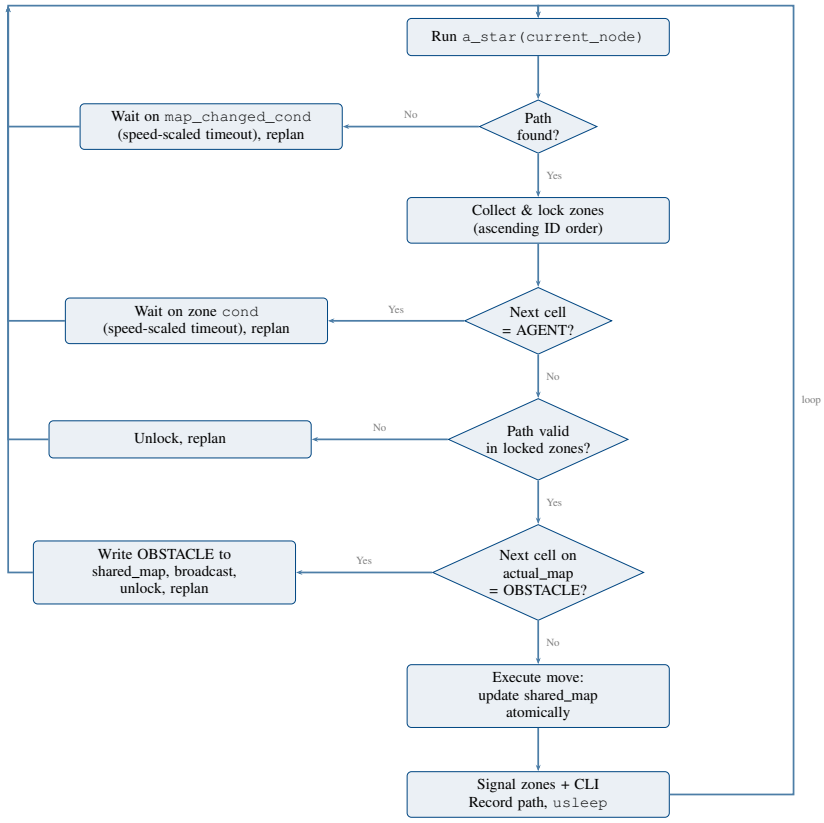

Fig. 3. Agent thread per-step decision flow.

TABLE IV
CONDITION VARIABLES.

Variable	Scope	Purpose
`zone.cond`	Per-zone	Wakes agents blocked on a cell in this zone after another agent moves.
`map_changed_cond`	Global	Broadcasts on obstacle discovery, waking all agents to replan.
`cli_cond`	Global	Signals the CLI thread to redraw after each agent move.

TABLE V
ATOMIC VARIABLES.

Variable	Type	Purpose
`cli_done`	`atomic_int`	Set to 1 by main after agents finish; CLI checks to exit.
`cli_update_seq`	`atomic_uint`	Counter incremented by agents; CLI compares to detect redraws.
`Node.type`	`_Atomic NodeType`	Atomic cell type prevents data races.

D. Condition Variables and Atomics

V. LIVELOCK MITIGATION

A. The Problem

The zone-based locking scheme described in Section IV-B guarantees freedom from *mutex deadlock*: no thread will block indefinitely waiting for a lock, because all locks are acquired in a fixed global order. However, the system remains susceptible to a distinct failure mode: *livelock*.

Consider two agents facing each other in a narrow corridor. Agent A's next cell is occupied by Agent B, and vice versa. Both agents enter the collision wait block, time out, and replan. Without any cost asymmetry, A* computes the same shortest path through the other agent (cost 1, identical to any other cell). Both agents arrive at the collision block again. The cycle repeats indefinitely: the agents are not blocked (the timed waits ensure they keep waking and replanning), but neither makes progress.

B. Design Space

Three approaches were considered for resolving multi-agent livelock within the project's decentralized, multithreaded architecture:

1) *Cooperative A**: Agents plan sequentially in priority order on a time-space graph, reserving cells at specific timesteps. This eliminates conflicts by construction but requires a centralized planner, which is fundamentally incompatible with the project's concurrent, independent-thread architecture.
2) *Priority-based yielding*: When two agents conflict, the higher-ID agent yields (waits without replanning) while the lower-ID agent proceeds. This provides deterministic conflict resolution for the common case. However, in a dead-end corridor with no alternative path, neither agent can make progress: the lower-ID agent replans through the same occupied cell, and the higher-ID agent waits

indefinitely. Resolving this requires an explicit retreat protocol where one agent backs away from the goal, which A* will never produce on its own.

3) *Agent-cell cost penalty*: Agent-occupied cells are assigned a traversal cost greater than 1 during A* planning. When an alternative route exists, the planner naturally routes around the occupied cell. When no alternative exists, the path still goes through the agent cell (at higher cost), and the existing collision detection and timed-wait mechanism in `runner.c` handles the runtime coordination.

C. Chosen Approach

The agent-cell cost penalty (option 3) was selected for several reasons. It requires no architectural changes, operating entirely within A*'s existing cost function. It is fully decentralized, with each agent making independent planning decisions. It degrades gracefully: when detours exist, agents route around each other; when they do not, the system falls back to the collision wait-and-replan mechanism.

The penalty value was set to $+2$ (total edge cost of 2 for agent-occupied cells vs. 1 for empty cells). This value was chosen so that A* prefers a 1-cell detour (cost 2: one extra step each way) over routing through an agent, but does not cause excessive detours. A higher penalty (e.g., $+10$) would force agents up to 5 cells out of their way even when a short alternative exists, inflating path lengths unnecessarily.

D. Impact on Optimality

The $+2$ penalty modifies A*'s edge cost function: $c(n, n') = 2$ when n' is agent-occupied, rather than the uniform $c = 1$. This means paths planned through or around agents are near-optimal with respect to the *current snapshot* of the shared map, but not strictly optimal with respect to the true static graph. Since agents move, the cost landscape is transient: the penalty reflects a planning-time heuristic for runtime conditions, not a permanent property of the graph.

Benchmark results (see companion report) show 3–12% path-length overhead over omniscient BFS across all tested configurations, confirming that the penalty introduces minimal excess cost in practice.

E. Known Limitation

The penalty does not guarantee livelock freedom in all topologies. In a narrow corridor with no alternative path, A* will still route through the occupied cell (it is the only option regardless of cost), and both agents will enter the wait-replan cycle. In practice, this scenario is rare on maps with reasonable obstacle density, and the timed waits ensure the system remains responsive rather than frozen. A complete solution would require either centralized planning (Cooperative A*, CBS) or an explicit retreat protocol, both of which represent significant architectural changes beyond the scope of this project.

TABLE VI
SPEED OPTIONS.

Level	Delay / step	`usleep` value
`slow`	500 ms	500 000 μs
`medium`	100 ms	100 000 μs
`fast`	20 ms	20 000 μs
`realtime`	0 ms	0 μs

VI. CLI AND VISUALIZATION

A. Invocation

- Interactive: `./astar` launches the ncurses menu.
- CLI: `./astar <file> [-speed <level>]` skips the menu.

B. Menu System

The file picker scans `input/` for `.txt` files and displays them with arrow-key navigation. Two special options allow entering a custom path or generating a random map (configurable rows, cols, agent count; ~15% obstacle density; agents capped at $\min(\text{rows} \times \text{cols}/10,\ 100)$). The speed picker presents the four levels from Table VI.

C. Rendering

TABLE VII
CELL TYPE RENDERING.

Cell Type	Symbol	Appearance
`EMPTY`	`.`	Dim white on default
`OBSTACLE`	(space)	White on white (solid)
`AGENT`	`A`	Bold white on red
`GOAL`	`G`	Bold black on green

Cell width adapts to terminal size (3, 2, or 1 characters). For maps exceeding the terminal even at width 1, arrow keys scroll a virtual camera. The rendering loop uses `pthread_cond_timedwait` (50ms timeout) to sleep until signalled, avoiding busy-waiting while staying responsive to keyboard input.

After all agents finish, a results screen shows elapsed time (`CLOCK_MONOTONIC`), per-agent step counts and full coordinate histories, with scrolling and a run-again/quit prompt.

VII. INPUT FORMAT AND MAP CONSTRUCTION

```
<rows> <cols>
<num_agents>
<agent_0_x> <agent_0_y>
<agent_1_x> <agent_1_y>
...
<goal_x> <goal_y>
<grid row rows-1>      (top row visually)
<grid row rows-2>
...
<grid row 0>           (bottom row visually)
```

Listing 6. Input file structure.

Grid characters: `0` = empty, `1` = obstacle. Coordinates are zero-indexed from the bottom-left. Grid rows are listed top-to-bottom visually (decreasing y).

The `init()` function in `map.c` parses dimensions, agents, and goal; builds `actual_map` with obstacle types and neighbour pointers; builds `shared_map` (all `EMPTY` except agents and goal, no neighbour pointers); and partitions into zones with mutex/cond initialization.

VIII. Memory Management

All dynamic memory follows strict allocation/deallocation pairing. The `path_history` buffer uses geometric doubling (initial capacity 64) for amortized $\mathcal{O}(1)$ appends.

TABLE VIII
MEMORY ALLOCATION AND DEALLOCATION.

Resource	Alloc	Dealloc
`actual_map` rows	`init()`	`cleanup_sim()`
`shared_map` rows	`init()`	`cleanup_sim()`
Agent array	`init()`	`cleanup_sim()`
`path_history`	`append_path()`	`cleanup_sim()`
Zone array	`init()`	`cleanup_sim()`
Thread handles	`main()`	`cleanup_sim()`
A* closed set	`a_star()`	before return
A* g-cost array	`a_star()`	before return
A* came-from	`a_star()`	before return
A* open set heap	`a_star()`	before return
PathNode list	`a_star()`	`free_path()`
Zone mutexes/conds	`init()`	`cleanup_sim()`

IX. Build System

```
CC = clang
CFLAGS = -g -Wall -Wextra -fcolor-diagnostics
         -fansi-escape-codes -I./include
LDFLAGS = -lm -lpthread -lncurses

ifdef ASAN
  CFLAGS  += -fsanitize=address
  LDFLAGS += -fsanitize=address
endif

SRC = src/main.c src/map.c src/astar.c src/runner
    .c
      src/cli.c src/heap.c src/types.c src/menu.c
TARGET = astar

$(TARGET): $(SRC)
    $(CC) $(CFLAGS) $(SRC) -o $(TARGET) $(LDFLAGS
        )

clean:
    rm -f $(TARGET)
```

Listing 7. Makefile.

Flags: `-g` for debug symbols, `-Wall -Wextra` for strict warnings, optional `make ASAN=1` for AddressSanitizer. Links `-lm`, `-lpthread`, `-lncurses`.

X. Summary

This simulator implements multi-agent A* pathfinding with fog of war, zone-based concurrency, and real-time ncurses visualization. The A* search supports both Manhattan and Euclidean heuristics, uses duplicate detection via a g-cost array to ensure correct path reconstruction, and employs a tuned $+2$ agent-cell penalty for decentralized livelock mitigation. Automated testing confirms single-agent BFS-optimal paths and multi-agent overhead under $1.5\times$. A companion comparison report presents the full 3,200-run scalability benchmark, which achieved a 100% success rate with 3–12% overhead across all grid sizes and agent counts.